

ZPD-Guided Program Repair from Student Submission Trajectories

Anonymous submission

Abstract

Large language model (LLM)의 발전으로 Automated Program Repair (APR)은 학생의 buggy program에 맞는 patch를 자동으로 생성할 수 있게 되었다. 그러나 current buggy program만으로는 같은 코드에 도달하기까지 학생이 수행한 modifications와 그에 따라 달라지는 적절한 repair 범위를 알기 어렵다. 본 연구는 학생의 submission trajectory에서 자연스럽게 이어지는 patch를 생성하는 ZPD-guided program repair task와 이를 위한 ZPDPatch를 제안한다. ZPDPatch는 실제 submission trajectories에서 서로 다른 개선 목표를 가진 repair transitions를 추출하고, 하나의 code LLM에 Progress, Strict, Answer QLoRA adapters를 독립적으로 학습한다. 추론에서는 세 adapters를 순차적으로 호출하고 candidate fixed program을 실행하여, Accepted에 도달하면 즉시 종료하며 그렇지 않으면 실행 개선과 modification size를 함께 고려해 patch를 선택한다. Project CodeNet 평가에서 ZPDPatch는 Seen과 Unseen problems에서 Zero-shot보다 각각 25.9와 10.0 percentage points 높은 Repair Rate를 보였고 더 작은 patches를 생성했으나, Seen에서 retrieval-based LSGen의 Repair Rate에는 미치지 못했다. 또한 full trajectory ablation은 current code only보다 일관된 우위를 보이지 않아, 성능 향상의 주된 근거가 trajectory history 자체보다는 adapter supervision과 sequential escalation에 있음을 확인했다.

Introduction

프로그래밍 교육에서 피드백은 학생이 결함을 인식하고 스스로 해결 전략을 조정하는 핵심 과정이다. 학생은 코드를 작성하고 제출하며, 자동 채점 시스템이 제공하는 실행 결과를 바탕으로 다시 코드를 수정한다. 온라인 저지와 autograder는 이러한 반복 과정을 즉각적으로 지원함으로써 교육의 확장성을 크게 높였지만, 학생에게 제공되는 정보는 여전히 제한적이다. 예를 들어 Wrong Answer, Runtime Error, Time Limit Exceeded와 같은 verdict는 제출 코드의 실패 여부를 알려줄 뿐, 동일한 실패 결과 안에서 학생의 수정이 실제로 개선되고 있는지, 같은 결함이 반복되고 있는지, 또는 현재 코드 흐름을 유지한 채 다음 modification step으로 나아갈 수 있는지는 구분하지 못한다. 결과적으로 학생은 같은 verdict를 여러 번 보면서도 자신의 코드에서 이어갈 수 있는 다음 modification step을 판단하기 어렵다. 또한, 학생마다 작성한

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

프로그램의 구조와 접근 방식이 다르기 때문에, 강사가 이를 모두 확인하고 일일이 개인화된 피드백을 제공하기란 어렵다.

Automated Program Repair (APR)은 buggy program의 결함을 수정하기 위한 patch를 자동으로 생성하는 technique이다. 학생마다 제출 코드의 구조와 결함 양상이 다르기 때문에, 이러한 patch는 각 학생 코드에 맞춘 실행 가능한 modifications이자 즉각적인 피드백으로 활용된다. CLARA는 다수의 reference programs를 클러스터링하고, 학생 코드가 가까운 reference program으로 이동할 수 있도록 프로그램 변환을 찾는다 (Gulwani, Radiček, and Zuleger 2018). SARFGEN은 학생 코드와 reference program 사이의 search-align-repair 절차를 통해 candidate patches를 생성한다 (Wang, Singh, and Su 2018). Refactory는 두 프로그램 사이의 구조적 차이를 refactoring 관점에서 정렬한다 (Hu et al. 2020). 최근에는 LLM 기반 patch generation, counterexample-guided repair, edit-driven retrieval enhancement를 통해 candidate patches를 생성하고 개선한다 (Zhang et al. 2022; Orvalho, Janota, and Manquinho 2025; Dai et al. 2026). 자동으로 생성된 patch는 강사의 피드백 부담을 줄이고, 학생이 자신의 buggy program과 fixed program을 비교하여 결함과 modifications를 학습할 기회를 제공한다.

Zone of Proximal Development (ZPD)¹는 학생이 도움 없이 수행할 수 있는 zone과 도움을 받더라도 아직 수행할 수 없는 zone 사이의, 적절한 도움을 받으면 도달할 수 있는 zone을 의미한다 (Cole et al. 1978). 프로그래밍 과제의 피드백도 단순히 correct patch를 제공하는 것보다 현재 학생의 ZPD를 고려한 patch가 더 적절할 수 있다. 예를 들어 같은 buggy program이더라도 여러 번의 제출 끝에 그 상태에 도달한 학생에게는 남은 결함을 한 번에 해결하는 correct patch보다, 약간의 도움을 통해 다음 단계에서 수행할 수 있는 patch가 더 적절할 수 있다. 반면 한 번의 제출만에 같은 상태에 도달한 학생에게는 correct patch가 이해 가능한 비교 대상이 될 수 있다. 따라서 patch의 적절성은 current code snapshot만으로 결정되지 않으며, 반복된 결함, 최근 modifications, verdict 변화와 같은 submission trajectory를 함께 고려해야 한다.

In this paper, 우리는 current buggy program만으로는 학생마다 달라지는 적절한 repair 범위를 결정하기 어렵다는 문제를 해결하기 위해 ZPDPatch를 제안한다. 먼저 실

¹https://en.wikipedia.org/wiki/Zone_of_proximal_development

제 student submission histories를 trajectories로 구성하고, verdict와 testcase outcome의 변화에 따라 Progress, Strict, Answer라는 세 repair transition 집합을 만든다. 다음으로 동일한 code LLM 위에 세 QLoRA adapters를 독립적으로 fine-tuning하여, 각각 부분적인 실행 개선, 명확한 verdict 개선, correct program 도달을 학습하게 한다. 추론에서는 adapters를 Progress, Strict, Answer 순으로 호출하며, 각 candidate fixed program을 실행한 뒤 Accepted이면 즉시 반환한다. 세 후보가 모두 실패하면 pass rate가 가장 높은 후보를 선택하고, 동물에서는 current buggy program과의 Tree Edit Distance (TED)가 가장 작은 후보를 선택한다. 이 구조는 하나의 correct patch를 모든 학생에게 바로 제공하는 대신, 학생의 trajectory와 현재 repair 가능성에 맞추는 도움을 범위를 점진적으로 확장한다.

본 논문의 기여는 다음과 같다.

- ZPDPatch operationalizes ZPD-guided APR by using the target student's own submission trajectory as the repair condition.
- We build a trajectory-conditioned repair framework that combines Progress, Strict, and Answer adapters for distinct repair scopes with execution-guided sequential escalation.
- We compare ZPDPatch with a zero-shot LLM and retrieval-based APR on seen and unseen problems under the same three-generation budget, evaluating pass rate, repair rate, improvement rate, repair time, and patch distance.

Related Work

Programming assignments를 위한 초기 APR systems는 reference programs를 clustering하거나 student code를 reference programs와 alignment하여 buggy programs를 repair 한다 (Gulwani, Radiček, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2020; Yi et al. 2017). 여기서 reference programs는 일반적으로 같은 programming assignment에 대한 peer students의 correct programs를 의미한다. 이후 연구는 assignment-aware neural feedback, multiple-function programs, diversified feedback generation으로 범위를 확장했다 (Heo et al. 2023; Choi, Heo, and Lee 2021; Choi and Lee 2025). 최근에는 LLM을 program repair engine으로 사용하는 연구가 증가하고 있다. RING은 multilingual LLM-based program repair를 연구하고 (Joshi et al. 2023), PyDex와 PyFiXV는 LLM과 retrieved examples를 사용한다 (Zhang et al. 2024; Phung et al. 2023). CREF는 multi-turn conversational repair를 사용하고 (Yang et al. 2024), FastFixer는 모델을 fine-tuning하며 (Liu et al. 2024), PAR은 peer students의 reference programs를 검색한다 (Zhao et al. 2024). Counterexample-guided repair는 zero-shot LLM과 MaxSAT-based Fault Localization을 결합하고 (Orvalho, Janota, and Manquinho 2025), LSGen은 submission 및 evaluation data를 활용한 iterative retrieval로 fixed programs를 생성한다 (Dai et al. 2026). 이들 접근은 주로 current buggy program, retrieved examples, 또는 auxiliary diagnostic information을 조건으로 사용한다.

Educational feedback은 학습자가 도움을 받아 productive 하게 수행할 수 있는 범위와도 맞아야 한다. 선행 연구는 programming exercises의 feedback 유형을 정리하고 (Keuning, Jeuring, and Heeren 2018), intelligent tutoring systems

에서 adaptive scaffolding과 help-seeking behavior를 연구했다 (Alevan and Koedinger 2000). Adaptive instructional systems에서는 학습 활동을 productive range 안에 유지하기 위해 ZPD를 computationally modeling했다 (Murray and Arroyo 2002; Chounta et al. 2017). Programming education의 next-step hints와 hint ladders는 단계적인 guidance를 제공한다 (Price, Dong, and Lipovac 2017; Roest, Keuning, and Jeuring 2024; Xiao, Hou, and Stamper 2024; Heickal and Lan 2024). To the best of our knowledge, ZPDPatch는 target student 자신의 submission trajectory를 직접 사용해 buggy program을 repair하는 최초의 APR tool이다.

Preliminaries

문제 p 에 대한 한 학생의 i 번째 submission을 $s_i = (c_i, v_i)$ 로 나타낸다. c_i 는 full source code이고 v_i 는 online judge verdict이다. 시간순으로 정렬되고 첫 Accepted submission에서 끝나는 $\tau = [s_1, s_2, \dots, s_n]$ 을 submission trajectory라 한다. 문제 설명과 실행 제약을 d_p , 공개 testcases를 $\mathcal{E}_p$ 라 한다. c_i 를 $\mathcal{E}_p$ 에서 재실행해 얻은 testcase별 verdict vector는 $o_i = (o_i(e))_{e \in \mathcal{E}_p}$ 로 나타낸다.

ZPDPatch의 한 repair example은 observed submissions의 sequence h , target full code y , 그리고 problem context d_p 로 구성된다. h 와 y 는 반드시 원 trajectory에서 인접할 필요가 없으며, adapter별 규칙이 trajectory에서 어떤 submissions를 유지하고 어느 code를 target으로 삼을지 결정한다. 모델은 $x = (d_p, h)$ 를 입력받아 complete program $\hat{c}$ 를 생성한다.

같은 current code라도 그 이전에 반복된 오류, 시도한 modifications, 그리고 개선 경로가 다르면 적절한 patch가 달라질 수 있다. 본 연구는 ZPD를 학생이 관찰된 trajectory에서 실제로 도달한 더 나은 상태로 operationalize한다. 즉 recorded next submission 자체를 모사하기보다, trajectory가 보여 주는 reachable improvement를 supervision으로 사용하여 current solution strategy를 이어 가는 patch를 학습한다.

ZPDPatch: ZPD-Guided Program Repair

Figure 1와 같이 ZPDPatch는 trajectory construction, adapter-specific supervision, independent adapter training, sequential repair의 네 단계로 구성된다. 세 adapters는 동일한 prompt와 base LLM을 공유하지만 서로 다른 규칙으로 구성된 datasets를 학습한다. 따라서 Progress, Strict, Answer라는 명칭은 prompt에 주어지는 역할 instruction이 아니라 training supervision의 차이를 나타낸다.

Trajectory Construction Submissions를 문제와 학생별로 묶어 시간순으로 정렬하고 첫 Accepted submission에서 trajectory를 종료한다. 각 code는 comments와 blank lines를 제거하고 indentation을 통일하며, 정규화 후 연속해서 동일한 submissions는 제거한다. 이 단계에서는 history 길이나 transition 수를 제한하지 않는다. 각 adapter의 규칙을 적용할 때까지 full code, online verdict, submission order를 모두 보존하며, Progress를 위해 모든 public testcases의 재실행 결과를 cache한다.

Adapter-Specific Supervision Verdict severity를 $\text{sev}(\text{AC}) = 0$, $\text{sev}(\text{WA}) = 1$, $\text{sev}(\text{TLE/MLE}) = 2$, $\text{sev}(\text{RE}) = 3$, $\text{sev}(\text{CE}) = 4$, 기타 internal error를 5로 둔다. 같은 non-empty testcase keys에서 모든 e 에 대해 $\text{sev}(o_t(e)) \leq \text{sev}(o_r(e))$ 이면 $o_t \preceq_{\text{sev}} o_r$ 로 쓴다.

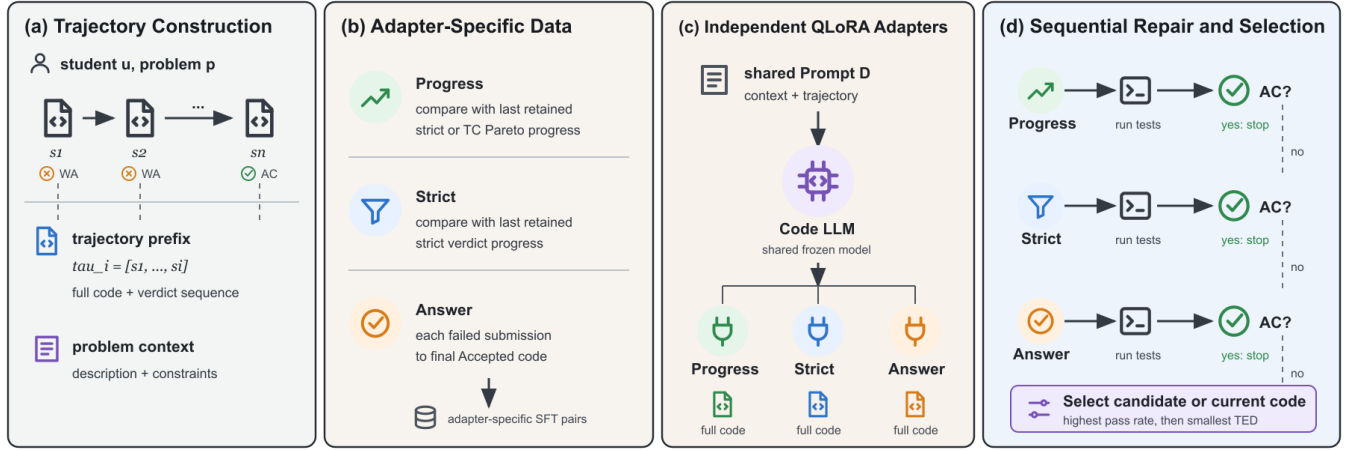


Figure 1: ZPDPatch overview. (a) 학생별 submission trajectory와 problem context를 구성한다. (b) adapter별 규칙으로 trajectory를 변환하고 supervision pairs를 만든다. (c) 동일한 code LLM 위에 세 QLoRA adapters를 독립적으로 학습한다. (d) adapters를 순차적으로 호출하고 실행 결과로 조기 종료하거나 후보를 선택한다.

Testcase-level progress는 한 testcase도 악화되지 않고 적어도 하나가 엄격히 개선되는 Pareto condition이다.

$$\text{TCProg}(r, t) = 1[o_t \preceq_{\text{sev}} o_r \wedge o_t \neq o_r].$$

Answer Adapter. Final submission s_n 이 Accepted이면, 그 앞의 모든 Non-AC submission을 각각 단독 input으로 사용하고 c_n 을 공통 target으로 둔다. 즉 $\tau = [s_1, \dots, s_9]$ 이면 $([s_1], c_9), \dots, ([s_8], c_9)$ 를 만든다. Answer는 trajectory를 자른 마지막 prefix 하나가 아니라, 각 실패 submission과 final answer 사이의 1:1 mapping을 학습한다.

Strict Adapter. $R = [s_1]$ 로 시작하고 original trajectory를 순서대로 scan한다. Candidate s_t 는 마지막 retained submission $s_r = R[-1]$ 보다 online verdict severity가 엄격히 낮을 때, 즉 $\text{sev}(v_t) < \text{sev}(v_r)$ 일 때만 R 에 추가한다. $R = [r_1, \dots, r_m]$ 을 얻으면 모든 $k = 2, \dots, m$ 에 대해 $([r_1, \dots, r_{k-1}], c_{r_k})$ 를 학습 example로 만든다. 따라서 비교 기준은 직전 original submission이 아니라 마지막 retained submission이다.

Progress Adapter. Strict와 같은 retained scan을 사용하되, s_t 를 (1) online verdict가 엄격히 개선되거나, (2) original online verdict가 $v_t = v_r$ 로 같고 $\text{TCProg}(r, t) = 1$ 일 때 유지한다. Retained trajectory에서 만드는 모든 prefix-target examples의 형식도 Strict와 같다. Progress는 online judge의 coarse verdict가 같더라도 testcase별 실행 결과가 단조롭게 개선된 실제 intermediate step을 추가로 학습한다.

세 datasets는 독립적으로 구성되며 서로 배타적일 필요가 없다. Strict examples는 Progress에도 포함될 수 있고 final Accepted transition도 두 datasets에 포함된다. 그러나 Answer의 pair construction과 Strict/Progress의 retained-prefix construction은 서로 다른 supervision objective를 형성한다.

Trajectory-Conditioned Adapter Training 세 adapters는 동일한 Prompt D를 사용한다. System message에는 problem description과 time/memory constraints를 제공하고, history의 각 retained submission은 full code와 online verdict를 포함하는 별도 user message로 표현한다. Cache가 존재하면 testcase별 execution feedback을 추가하고, 연속 sub-

missions 사이의 AST/line edit summary도 함께 제공한다. 마지막 user message가 current buggy program이며 assistant target은 하나의 complete Python program이다. Prompt에는 adapter 역할을 명시하지 않으므로 specialization은 오직 dataset construction에서 발생한다.

Problem context와 trajectory prefix를 $x_i = (d_p, \tau_j^{(i)})$, target code token sequence를 y_i 라 할 때, 각 adapter $a \in \{\text{prog}, \text{strict}, \text{ans}\}$ 는 자신의 dataset $\mathcal{D}_a$ 에서 standard supervised fine-tuning (SFT) objective를 최소화한다.

$$\mathcal{L}_a = -\frac{1}{|\mathcal{D}_a|} \sum_{(x_i, y_i) \in \mathcal{D}_a} \frac{1}{|y_i|} \sum_{k=1}^{|y_i|} \log P_{\theta, \phi_a}(y_{i,k} | y_{i, < k}, x_i),$$

여기서 θ 는 공유하는 frozen base LLM이고 ϕ_a 는 adapter별 QLoRA parameters이다. Loss는 assistant target tokens에만 적용하고 prompt tokens는 masking한다. 세 adapters는 하나의 multi-head model로 공동 학습되지 않으며 동일한 base LLM 위에 별도 checkpoints로 저장된다.

Sequential Repair and Selection 추론 시 current program과 trajectory의 모든 submissions를 먼저 public testcases에서 실행하여 Prompt D의 execution feedback을 완성한다. 세 adapters는 동일한 original trajectory prefix를 기반으로 Progress, Strict, Answer 순서로 각각 하나의 full-code candidate를 생성한다. Candidate가 AC이면 즉시 반환한다. 실패하면 verdict, pass rate, testcase별 outcomes, current program 대비 개선 여부를 다음 단계 prompt에 추가하여 같은 실패를 반복하지 않도록 한다. No-op 또는 regression candidate는 code를 재제시하지 않고 실행 결과만 전달한다.

세 candidates가 모두 실패하면 current buggy program c_i 를 fallback으로 포함한 집합 $\mathcal{C} = \{c_i, c_i^{\text{prog}}, c_i^{\text{strict}}, c_i^{\text{ans}}\}$ 에서 최종 program을 선택한다. $\text{Pass}(c) = |Q(c)|/|\mathcal{E}_p|$ 이고 ted가 AST Tree Edit Distance일 때 선택 규칙은 다음과 같다.

$$c^* = \arg \max_{c \in \mathcal{C}} (\text{Pass}(c), -\text{ted}(c_i, c)).$$

따라서 selector는 먼저 가장 많은 testcases를 통과한 program을 고르고, 동물에서는 학생의 current code를 가장 적게 바꾸는 program을 선택한다. Current code보다 개선되지 않은 candidate만 존재하면 TED가 0인 current code가 선택되어 system이 abstain한다.

Experimental

Research Questions 평가는 다음 다섯 research questions를 다룬다.

- **RQ1: Repair Effectiveness.** ZPDPatch는 baselines보다 buggy programs를 얼마나 효과적으로 repair하는가?
- **RQ2: Trajectory Conditioning.** Submission trajectory conditioning은 program repair에 어떤 영향을 미치는가?
- **RQ3: Adapter Specialization.** Progress, Strict, Answer adapters는 repair behavior에서 어떻게 다른가?
- **RQ4: Sequential Escalation.** Sequential escalation은 repair effectiveness와 efficiency에 어떤 영향을 미치는가?
- **RQ5: Problem Generalization.** ZPDPatch는 unseen problems에 얼마나 잘 generalize하는가?

Group	Role	Prob.	Tests	Traj.	Sub.	Prefix	Users
Seen	Train / LSGen DB			14,638	58,872	44,234	4,072
	Valid	492	9,446	1,830	7,283	5,453	1,283
	Test			1,830	7,068	5,238	1,291
Unseen	Test	54	642	260	965	705	237

Table 1: 4,096-token whole-trajectory filter 이후 volume-ordered 90:10 problem split의 dataset 통계. Seen의 Prob.와 Tests는 세 trajectory splits가 공유하는 전체 problem group을 나타낸다. Prefix는 adapter별 filtering 전 생성 가능한 raw prefixes 수이며, users는 Seen splits 사이에서 중복될 수 있다.

Dataset Construction and Split Project CodeNet (Puri et al. 2021)의 Python800 benchmark에 포함된 problems를 기준으로 Python submissions, problem descriptions, meta-data, testcases를 결합한다. 연속된 normalized-code duplicates를 제거하고, 첫 Accepted 이전에 하나 이상의 Non-Accepted submission이 있으며 전체 trajectory가 최소 세 submissions를 포함하는 경우만 유지한다. 첫 Accepted 이후 submissions는 제거하고, 마지막 Accepted code가 Python800 benchmark에서 검증된 correct program인 trajectories만 남긴다. 또한 최소 세 problems에 나타나는 users와 최소 두 trajectories를 가진 problems만 사용한다. 이 refined corpus는 546 problems, 21,454 trajectories, 99,432 submissions, 10,088 testcases로 구성된다.

Split 전에 Qwen tokenizer와 Prompt D로 Answer, Strict, Progress envelope, final evaluation에서 가능한 모든 prompt-target configuration의 total tokens를 측정한다. Progress envelope는 실제 testcase outcomes와 무관하게 verdict가 같은 candidate를 모두 유지할 수 있다고 가정하는 conservative upper bound이다. 어느 configuration에서든 prompt와 target의 합이 4,096 tokens를 초과하면 해당 example의 끝부분을 자르지 않고 trajectory 전체를 제외한다. 이 규칙으로 2,896 trajectories를 제외하고 18,558 trajectories

를 분할한다. 따라서 남은 data에는 별도의 max_history, 최대 transition 수, code/testcase truncation을 적용하지 않는다.

Table 1과 같이 problems를 적격 trajectory 수의 내림차순으로 정렬하고 상위 90%인 492 problems를 Seen, 나머지 54 problems를 Unseen으로 배정한다. 이는 충분한 trajectories가 있는 problems로 모델과 retrieval database를 학습하면서, 학습에서 완전히 제외된 low-resource problems에서 generalization을 평가하기 위한 설정이다. Seen의 각 problem에서 Train, Valid, Test에 최소 한 trajectory씩 보존한 뒤 전체 trajectories를 deterministic 80:10:10으로 나눈다. Unseen의 260 trajectories는 분할하거나 학습에 노출하지 않고 모두 Test에 사용한다.

Seen Train은 ZPDPatch fine-tuning과 LSGen의 same-problem retrieval database에 사용하고, Seen Valid는 model 및 prompt configuration을 결정하는 데 사용한다. RQ1은 Seen Test에서 ZPDPatch, Zero-shot, LSGen을 비교한다. RQ5는 Unseen Test에서 ZPDPatch와 Zero-shot을 비교한다. LSGen은 query problem의 peer correct programs를 요구하므로, same-problem data를 노출하면 problem-held-out 조건이 깨지는 Unseen Test에는 적용하지 않는다.

Training examples는 Seen Train trajectories에 Answer, Strict, Progress의 adapter-specific construction을 각각 적용하여 만든다. Seen Valid도 동일한 규칙으로 독립적으로 변환하며 training에 포함하지 않는다. Final evaluation에서는 $[s_1, \dots, s_{n-1}]$ 을 observed trajectory, s_{n-1} 을 current buggy submission, 마지막 Accepted code c_n 을 공통 oracle c^+ 로 둔다. Public testcases에서 current program은 Non-AC이고 oracle은 AC인 trajectories만 포함하여 모든 approaches가 같은 repair instances와 correctness oracle을 사용하게 한다.

Baselines Zero-shot은 ZPDPatch와 같은 Qwen base model과 Prompt D를 사용하지만 fine-tuning하지 않는다. 첫 candidate가 실패하면 candidate verdict와 통과 testcase 수를 conversation에 추가하여 최대 세 번 repair한다. 따라서 trajectory prompt 자체의 효과와 adapter training의 추가 효과를 구분한다.

LSGen (Dai et al. 2026)은 공개 artifact의 repair-pair filtering, edit-vector retrieval, diff-guided generation, iterative re-retrieval을 동일 dataset과 local judge에 연결한다. Seen Test의 각 query와 같은 problem의 Seen Train trajectories에서 buggy/correct pairs를 구성하고 top-5 pairs를 검색하며 최대 세 iterations를 수행한다. Public artifact가 dataset, API, judge paths를 고정하므로 infrastructure만 공통 runner로 교체하고 핵심 retrieval 및 generation 절차는 유지한다. LSGen은 target student’s trajectory를 사용하지 않는다.

세 approaches는 같은 base LLM, testcases, timeout, 최대 세 번의 candidate generation budget을 사용한다. 모든 generated patches는 공통 JSONL format으로 저장하며 patch index, source, full code, verdict, pass rate, testcase별 outcomes, generation 및 execution time을 기록한다.

Implementation

Details Qwen2.5-Coder-7B-Instruct (Hui et al. 2024)를 base model로 사용하고 canonical-v5 Seen Train에서 세 4-bit QLoRA (Dettmers et al. 2023) adapters를 각각 1 epoch 학습한다. Base weights는 NF4 double quantization으로 고정하고 BF16 computation을 사용한다. LoRA는 q, k, v, o attention projections와 gate/up/down MLP projections에 적용하며 rank 16, scaling factor 32, dropout 0.05로 설정한다.

Optimization은 paged AdamW 8-bit, learning rate 2×10^{-4} , cosine schedule, warmup ratio 0.03을 사용한다. Gradient checkpointing과 length-grouped batching을 활성화하고, micro-batch size 2와 gradient accumulation 8로 effective batch size 16을 유지하며 seed는 2027이다. Validation은 epoch 끝에 한 번 수행하고 가장 낮은 validation loss의 checkpoint를 사용한다. Generation은 batch size 1의 greedy decoding이며 고정 output-token 상한 대신 input 뒤에 남은 model context 전체를 사용한다. 모든 학습과 generation은 NVIDIA RTX 5090 한 장에서 수행한다.

모든 original submissions와 generated candidates는 같은 Python runner로 전체 public testcases에서 실행한다. Runner는 testcase당 2.5초 timeout과 2GB memory limit을 적용하고 testcase별 verdict와 pass rate를 cache한다. 이 cache는 Progress dataset construction, sequential stage feedback, candidate selection, 최종 evaluation에서 동일하게 재사용한다.

Ablation Protocols RQ2에서는 Strict와 Progress의 adapter-specific rules로 Seen과 Unseen Test examples를 구성한다. Full Trajectory는 retained prefix 전체를 사용하고, Current Code Only는 동일한 examples, targets, hyperparameters로 adapters를 다시 학습하되 current submission 이전 history를 제거한다. Current Code Only의 position은 S_1 로 재설정하여 original index가 trajectory 길이를 누출하지 않게 한다. Answer는 원래부터 각 example이 한 submission만 입력하므로 이 ablation에서 제외한다.

RQ3에서는 동일한 final-evaluation trajectories에서 Progress, Strict, Answer를 각각 단독으로 한 번 실행하여 완전 repair, 부분 개선, modification size의 차이를 비교한다. RQ4에서는 이 세 단독 실행과 Progress-Strict-Answer 순의 stage feedback, AC early stopping, fallback selector를 모두 적용하는 full Sequential을 비교한다. RQ5는 RQ1과 같은 full Sequential configuration을 problem-held-out Unseen Test에 그대로 적용한다.

Evaluation Metrics 평가 sample 수를 M , m 번째 buggy program과 fixed program을 c_m 과 $\hat{c}_m$ 이라 하자. $\text{Pass}(c)$ 는 program c 가 해당 problem의 testcases 중 통과한 비율이다. 모든 rate는 sample별 score $z_m \in [0, 1]$ 의 평균으로 정의한다.

$$\text{Rate}(z) = \frac{1}{M} \sum_{m=1}^M z_m.$$

Pass Rate (PR)는 $z_m = \text{Pass}(\hat{c}_m)$, Repair Rate (RR)는 $z_m = \mathbf{1}[\text{Pass}(\hat{c}_m) = 1]$, Improvement Rate (IR)는 $z_m = \mathbf{1}[\text{Pass}(\hat{c}_m) > \text{Pass}(c_m)]$ 으로 계산한다. PR은 fixed program이 통과한 testcase 비율, RR은 모든 testcases를 통과한 fixed program의 비율, IR은 buggy program보다 더 많은 testcases를 통과한 fixed program의 비율을 나타낸다.

Average Time Taken (ATT)는 approach별로 한 problem의 전체 buggy programs를 repair하기 직전부터 candidate generation, execution, selection이 모두 끝날 때까지 wall-clock time Δt_p 를 측정한 뒤 전체 buggy program 수로 나눈다. 모든 approaches가 공유하는 buggy/oracle execution cache 구축, model loading, training, LSGen retrieval database의 construction은 offline preparation으로 제외한다.

TED는 repair에 성공한 programs 중 Python AST parsing이 가능한 집합 R 에서만 계산한다. $\text{TED}_{B \rightarrow F}$ 는 buggy

program에서 fixed program까지의 modification size이고, $\text{TED}_{F \rightarrow O}$ 는 fixed program과 recorded oracle c_m^+ 의 구조적 차이이다.

RQ2의 $\text{TED}_{F \rightarrow T}$ 는 repair에 성공하고 AST parsing이 가능한 samples에서 generated program과 adapter-specific construction이 선택한 recorded target c_m^T 사이의 평균 TED이다. Full Trajectory와 Current Code Only는 example과 target이 동일하므로 이 값은 history conditioning이 recorded reachable improvement에 얼마나 가까운 correct patch를 유도하는지 비교한다.

RR과 IR에는 Wilson 95% confidence interval을 보고한다. 같은 buggy programs에 대한 approaches의 RR 차이는 exact McNemar test로 검정한다. PR과 TED는 paired samples에서 10,000회 bootstrap confidence interval을 계산한다.

Results

RQ1: Repair Effectiveness.

Approach	PR	RR	IR	ATT	$\text{TED}_{B \rightarrow F}$	$\text{TED}_{F \rightarrow O}$
Seen						
Zero-shot	76.3	30.7	43.7	12.87	27.25	27.67
LSGen	88.0	80.2	82.4	13.00	88.27	83.23
ZPDPatch	84.9	56.6	64.6	12.16	19.71	21.89
Unseen						
Zero-shot	75.3	62.0	68.8	3.89	17.46	15.80
ZPDPatch	83.0	72.0	76.8	6.17	11.66	12.54

Table 2: RQ1의 Seen 결과와 RQ5의 Unseen 결과. TED는 repair에 성공하고 AST parsing이 가능한 samples에서만 계산한다.

Seen problems에서 Zero-shot은 PR 76.3%, RR 30.7%, IR 43.7%를 기록한 반면, ZPDPatch는 각각 84.9%, 56.6%, 64.6%를 기록했다. RR과 IR의 Wilson 95% CI는 Zero-shot에서 [27.9, 33.6]와 [40.7, 46.8], ZPDPatch에서 [53.5, 59.6]과 [61.6, 67.5]였다. 동일한 997 samples 중 ZPDPatch만 repair한 경우는 301개, Zero-shot만 repair한 경우는 43개였으며, exact McNemar test에서 차이가 유의했다($p = 8.20 \times 10^{-49}$). ZPDPatch의 PR도 8.57 percentage points 높았다(95% bootstrap CI [6.99, 10.22]). 따라서 adapter training과 sequential execution을 결합한 ZPDPatch는 같은 base LLM의 trajectory-prompted Zero-shot보다 완전 repair와 부분 개선을 모두 증가시켰다.

LSGen은 PR 88.0%, RR 80.2%, IR 82.4%로 가장 높은 execution correctness를 보였다. ZPDPatch와의 paired comparison에서는 LSGen만 repair한 경우가 306개, ZPDPatch만 repair한 경우가 70개였고, RR 차이는 유의했다($p = 2.70 \times 10^{-36}$). ZPDPatch의 PR은 LSGen보다 3.14 percentage points 낮았다(95% CI [-5.14, -1.08]). 반면 ZPDPatch의 $\text{TED}_{B \rightarrow F}$ 는 19.71로 Zero-shot의 27.25와 LSGen의 88.27보다 작았다. 두 방법이 모두 repair한 samples에서도 ZPDPatch의 TED는 Zero-shot보다 평균 8.07 낮았고(95% CI [-12.14, -4.53]), LSGen보다 43.41 낮았다(95% CI [-48.94, -38.20]). ZPDPatch의 ATT는 12.16초로 세 approaches 중 가장 짧았다. 이 결과는 same-problem retrieval이 correct patch coverage에 강한 반면, ZPDPatch는 더 작은 modification으로 상당수의 programs를 repair함을 보여준다.

Answer to RQ1. ZPDPatch는 Zero-shot보다 더 많은 buggy programs를 repair하고 개선하면서 더 작은 patch를 생성했지만, LSGen보다 correct patch coverage는 낮았다. 즉, ZPDPatch는 solution retrieval을 대체하기보다 retrieval database 없이 작은 repair를 생성하는 complementary approach이다.

RQ2: Trajectory Conditioning.

Split	Adapter	Input	PR	RR	IR	TED _{F→T}
Seen	Strict	Current	56.8	31.2	54.6	40.23
	Strict	Full	57.5	30.5	53.3	41.05
	Progress	Current	58.4	26.3	49.7	36.23
	Progress	Full	57.4	25.9	49.1	33.15
Unseen	Strict	Current	66.7	56.2	71.4	20.20
	Strict	Full	65.4	53.4	70.8	19.27
	Progress	Current	65.7	52.1	66.4	17.63
	Progress	Full	66.5	53.7	68.3	18.16

Table 3: RQ2 trajectory conditioning 결과. Seen Strict/Progress의 N 은 2,151/2,674, Unseen은 322/378이다.

Full Trajectory는 Current Code Only와 비교해 Seen Strict에서 PR만 0.7 points 높았고 RR과 IR은 각각 0.7과 1.3 points 낮았다. Seen Progress에서도 PR, RR, IR이 각각 1.0, 0.4, 0.6 points 낮았다. Unseen에서는 Strict의 세 execution metrics가 낮아진 반면 Progress는 각각 0.8, 1.6, 1.9 points 높아져 방향이 다시 엇갈렸다. 네 adapter-split 비교의 RR 차이는 모두 exact McNemar test에서 유의하지 않았다($p \geq 0.078$). Target TED도 Full Trajectory가 Seen Progress와 Unseen Strict에서는 작았지만 나머지 두 조건에서는 컸다. **Answer to RQ2.** 현재 실험에서는 submission history를 추가하는 것만으로 repair 성능이 일관되게 향상되지 않았다. 따라서 RQ1의 개선을 trajectory history 자체의 인과 효과로 해석할 수 없으며, adapter-specific supervision과 sequential escalation을 포함한 전체 구성의 효과로 한정해야 한다.

RQ3: Adapter Specialization.

Adapter	PR	RR	IR	TED _{B→F}	TED _{F→O}
Progress	73.4	44.8	52.0	15.85	15.89
Strict	74.8	44.5	51.2	17.45	18.38
Answer	77.5	50.1	57.5	20.22	20.61

Table 4: RQ3 adapter specialization 결과(Seen, $N = 997$).

Answer는 가장 높은 PR, RR, IR을 기록했지만 TED_{B→F}도 20.22로 가장 컸다. 반대로 Progress는 RR이 44.8%로 낮았으나 TED가 15.85로 가장 작아, 더 제한적인 modification을 생성하는 경향을 보였다. Progress와 Strict의 RR 차이는 유의하지 않았지만($p = 0.857$), Answer의 RR은 Progress와 Strict보다 각각 5.2와 5.5 points 높았다(각각 $p = 3.11 \times 10^{-4}$, $p = 6.40 \times 10^{-5}$).

Answer to RQ3. Adapter supervision은 repair scope의 차이로 나타났다. Answer는 더 많은 programs를 완전히 repair하는 대신 큰 patch를 생성했고, Progress는 가장 작은 patch를 생성했다. 다만 Strict는 Progress와 뚜렷하게 구분되는 repair rate를 보이지 않았다.

RQ4: Sequential Escalation.

Configuration	PR	RR	IR	ATT	#Patch
Progress	73.4	44.8	52.0	10.91	1.00
Strict	74.8	44.5	51.2	10.95	1.00
Answer	77.5	50.1	57.5	9.99	1.00
Sequential	84.9	56.6	64.6	12.16	2.05

Table 5: RQ4 sequential escalation 결과.

Sequential은 가장 강한 single adapter인 Answer보다 PR 7.4 points, RR 6.5 points, IR 7.1 points 높았다. 동일 samples에서 Sequential만 repair한 경우는 119개, Answer만 repair한 경우는 54개였다($p = 8.57 \times 10^{-7}$). Progress와 Strict에 대해서도 Sequential의 RR 우위가 유의했다(각각 $p = 5.46 \times 10^{-23}$, $p = 9.36 \times 10^{-22}$). AC early stopping으로 997 samples 중 439개가 Progress에서, 추가 66개가 Strict에서 종료되어 평균 generation 수는 최대 3회보다 작은 2.05회였고, ATT 증가는 Answer 대비 2.17초에 그쳤다.

Answer to RQ4. Execution-guided sequential escalation은 single adapters보다 더 많은 programs를 repair하고 개선했다. Early stopping은 평균 generation 수를 2.05회로 줄여, 세 adapters를 항상 모두 실행하지 않고도 coverage를 높였다.

RQ5: Problem Generalization Unseen 250 samples에서 ZPDPatch는 PR 83.0%, RR 72.0%, IR 76.8%를 기록해 Zero-shot의 75.3%, 62.0%, 68.8%를 모두 앞섰다. ZPDPatch만 repair한 경우는 42개, Zero-shot만 repair한 경우는 17개였으며 RR 차이는 유의했다($p = 0.00155$). PR 차이는 7.65 percentage points였다(95% bootstrap CI [3.59, 11.93]). 동시에 ZPDPatch의 TED_{B→F}는 11.66으로 Zero-shot의 17.46보다 작았다. 두 방법이 모두 repair한 parseable samples에서도 ZPDPatch의 TED가 평균 6.33 작았다(95% CI [-9.77, -3.31]).

Answer to RQ5. ZPDPatch의 Zero-shot 대비 이점은 학습에서 완전히 제외된 problems에서도 유지되었다. 다만 Unseen은 low-resource problems로 구성되므로 Seen과 Unseen의 절대 성능 차이를 problem familiarity의 효과로 직접 해석하지 않는다.

Conclusion

본 연구는 current buggy program과 그 코드에 도달한 submission trajectory를 조건으로 patch를 생성하는 ZPDPatch를 제안했다. Seen 평가에서 ZPDPatch는 Zero-shot보다 RR을 25.9 percentage points 높이고 더 작은 patch를 생성했으며, 이 이점은 Unseen problems에서도 유지되었다. Progress, Strict, Answer의 순차 실행은 가장 강한 single adapter보다 RR을 6.5 points 높였고, early stopping으로 평균 2.05회만 생성했다. 반면 Current Code Only ablation에서 Full Trajectory의 일관된 우위는 관찰되지 않았다. 따라서 현재 증거는 adapter-specific reachable-improvement supervision과 execution-guided escalation의 효과를 지지하지만, submission history 자체가 성능 향상의 원인이라는 주장은 지지하지 않는다. 향후에는 history가 실제로 유용한 trajectory 조건을 식별하고, 동일 current code에 서로 다른 histories를 결합한 controlled evaluation으로 trajectory conditioning의 인과 효과를 검증해야 한다.

References

Aleven, V.; and Koedinger, K. R. 2000. Limitations of student control: Do students know when they need help? In *Internat-*

- tional conference on intelligent tutoring systems, 292–303. Springer.
- Choi, D.; Heo, J.; and Lee, E. 2021. Automated Feedback Generation for Multiple Function Programs. In *2021 28th Asia-Pacific Software Engineering Conference (APSEC)*, 582–583. IEEE.
- Choi, D.; and Lee, E. 2025. Automated Feedback Generation for Programming Assignments Through Diversification. In *2025 IEEE/ACM 37th International Conference on Software Engineering Education and Training (CSEE&T)*, 230–241. IEEE.
- Chounta, I.-A.; Albacete, P.; Jordan, P.; Katz, S.; and McLaren, B. M. 2017. The “Grey Area”: A computational approach to model the Zone of Proximal Development. In *European conference on technology enhanced learning*, 3–16. Springer.
- Cole, M.; John-Steiner, V.; Scribner, S.; and Souberman, E. 1978. Mind in society. *Mind in society the development of higher psychological processes*. Cambridge, MA: Harvard University Press.
- Dai, Z.; Zhao, Z.; Wang, H.; Tang, X.; Wu, S.; Yao, C.; Gao, Z.; and Chen, J. 2026. Learner-Tailored Program Repair: A Solution Generator with Iterative Edit-Driven Retrieval Enhancement. *arXiv preprint arXiv:2601.08545*.
- Dettmers, T.; Pagnoni, A.; Holtzman, A.; and Zettlemoyer, L. 2023. Qlora: Efficient finetuning of quantized llms. *Advances in neural information processing systems*, 36: 10088–10115.
- Gulwani, S.; Radiček, I.; and Zuleger, F. 2018. Automated clustering and program repair for introductory programming assignments. *ACM SIGPLAN Notices*, 53(4): 465–480.
- Heickal, H.; and Lan, A. 2024. Generating feedback-ladders for logical errors in programming using large language models. *arXiv preprint arXiv:2405.00302*.
- Heo, J.; Jeong, H.; Choi, D.; and Lee, E. 2023. Referent: Transformer-based feedback generation using assignment information for programming course. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*, 101–106. IEEE.
- Hu, Y.; Ahmed, U. Z.; Mechtaev, S.; Leong, B.; and Roychoudhury, A. 2020. Re-factoring based program repair applied to programming assignments. In *Proceedings-2019 34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019*, 388–398. IEEE.
- Hui, B.; Yang, J.; Cui, Z.; Yang, J.; Liu, D.; Zhang, L.; Liu, T.; Zhang, J.; Yu, B.; Lu, K.; et al. 2024. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*.
- Joshi, H.; Sanchez, J. C.; Gulwani, S.; Le, V.; Verbruggen, G.; and Radiček, I. 2023. Repair is nearly generation: Multilingual program repair with llms. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, 5131–5140.
- Keuning, H.; Jeuring, J.; and Heeren, B. 2018. A systematic literature review of automated feedback generation for programming exercises. *ACM Transactions on Computing Education (TOCE)*, 19(1): 1–43.
- Liu, F.; Liu, Z.; Zhao, Q.; Jiang, J.; Zhang, L.; Sun, Z.; Li, G.; Li, Z.; and Ma, Y. 2024. Fastfixer: An efficient and effective approach for repairing programming assignments. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 669–680.
- Murray, T.; and Arroyo, I. 2002. Toward measuring and maintaining the zone of proximal development in adaptive instructional systems. In *International conference on intelligent tutoring systems*, 749–758. Springer.
- Orvalho, P.; Janota, M.; and Manquinho, V. M. 2025. Counterexample guided program repair using zero-shot learning and maxsat-based fault localization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 649–657.
- Phung, T.; Cambronero, J.; Gulwani, S.; Kohn, T.; Majumdar, R.; Singla, A.; and Soares, G. 2023. Generating high-precision feedback for programming syntax errors using large language models. *arXiv preprint arXiv:2302.04662*.
- Price, T. W.; Dong, Y.; and Lipovac, D. 2017. iSnap: towards intelligent tutoring in novice programming environments. In *Proceedings of the 2017 ACM SIGCSE Technical Symposium on computer science education*, 483–488.
- Puri, R.; Kung, D. S.; Janssen, G.; Zhang, W.; Domeniconi, G.; Zolotov, V.; Dolby, J.; Chen, J.; Choudhury, M.; Decker, L.; et al. 2021. Project codenet: A large-scale ai for code dataset for learning a diversity of coding tasks. *arXiv preprint arXiv:2105.12655*, 1035.
- Roest, L.; Keuning, H.; and Jeuring, J. 2024. Next-step hint generation for introductory programming using large language models. In *Proceedings of the 26th Australasian Computing Education Conference*, 144–153.
- Wang, K.; Singh, R.; and Su, Z. 2018. Search, align, and repair: data-driven feedback generation for introductory programming exercises. In *Proceedings of the 39th ACM SIGPLAN conference on programming language design and implementation*, 481–495.
- Xiao, R.; Hou, X.; and Stamper, J. 2024. Exploring how multiple levels of GPT-generated programming hints support or disappoint novices. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, 1–10.
- Yang, B.; Tian, H.; Pian, W.; Yu, H.; Wang, H.; Klein, J.; Bissyandé, T. F.; and Jin, S. 2024. Cref: An llm-based conversational software repair framework for programming tutors. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 882–894.
- Yi, J.; Ahmed, U. Z.; Karkare, A.; Tan, S. H.; and Roychoudhury, A. 2017. A feasibility study of using automated program repair for introductory programming assignments. In *Proceedings of the 2017 11th joint meeting on foundations of software engineering*, 740–751.
- Zhang, J.; Cambronero, J.; Gulwani, S.; Le, V.; Piskac, R.; Soares, G.; and Verbruggen, G. 2022. Repairing bugs in python assignments using large language models. *arXiv preprint arXiv:2209.14876*.
- Zhang, J.; Cambronero, J. P.; Gulwani, S.; Le, V.; Piskac, R.; Soares, G.; and Verbruggen, G. 2024. Pydex: Repairing

bugs in introductory python assignments using llms. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1): 1100–1124.

Zhao, Q.; Liu, F.; Zhang, L.; Liu, Y.; Yan, Z.; Chen, Z.; Zhou, Y.; Jiang, J.; and Li, G. 2024. Peer-aided repairer: Empowering large language models to repair advanced student assignments. *arXiv preprint arXiv:2404.01754*.